SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Debugging or Optimization task

Code of Conduct:

I M. Pranay (192525382)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M. Pranay

SAMPLE QUESTIONS

(QUESTIONS CAN BE CHANGED AS PER YOUR REQUIREMENTS)

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(20 Marks) (BL4)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(20 Marks) (BL4)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(20 Marks) (BL4)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(20 Marks) (BL4)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.
Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(20 Marks)(BL4)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

TOTAL MARKS AWARDED **FACULTY SIGNATURE**

ANSWERS

Q1. Deadlock Debugging (Code Trace)

Question: Identify the exact condition that causes deadlock. Suggest an optimization to eliminate the deadlock without changing process logic.

1. Problem Identification

Deadlock occurs when two or more processes wait indefinitely for resources held by each other. In the given program, P1 locks R1 first, while P2 locks R2 first.

2. Root Cause

- P1 acquires R1 and waits for R2.
- P2 acquires R2 and waits for R1.
- P1 cannot proceed until P2 releases R2.
- P2 cannot proceed until P1 releases R1.

This creates a circular wait, resulting in deadlock.

3. Four Conditions of Deadlock

1. Mutual Exclusion: Only one process can use a resource at a time.
2. Hold and Wait: A process holds one resource while waiting for another.
3. No Preemption: Resources cannot be forcibly taken away.
4. Circular Wait: Processes wait for resources held by one another in a circular chain.

4. Optimized Code

The deadlock can be prevented by acquiring resources in the same order in both processes.

```
// Process P1
lock(R1);
lock(R2);

// Critical section

unlock(R2);
unlock(R1);

// Process P2
lock(R1);
lock(R2);

// Critical section
```

unlock(R2);
unlock(R1);

5. Optimization Explanation

- Both processes acquire R1 before R2.
- Circular wait is eliminated.
- Resources are released in reverse order.
- The critical section executes without this lock-order deadlock.

Conclusion

Using a consistent resource-locking order eliminates circular wait and prevents deadlock, provided all processes follow the same order.

Q2. CPU Utilisation Problem

Question: A system shows low CPU utilization of 30%, even though processes are ready. Debug the cause and propose a system-level optimization.

1. Problem Identification

CPU utilization is low because processes frequently transition between the ready and waiting states. This may indicate excessive I/O blocking, synchronization delays, or inefficient process scheduling.

2. Root Cause Analysis

The observed transitions are:

Ready → Waiting → Ready → Waiting → Ready

- A process enters the ready state and waits for CPU execution.
- It may perform an I/O operation or wait for a resource.
- The process enters the waiting state.
- After the operation completes, it returns to the ready state.
- Frequent blocking can leave the CPU idle if no other process is runnable.

3. Debugging Steps

1. Examine scheduler logs to identify frequent waiting-state transitions.
2. Check whether processes are blocked by I/O operations.
3. Identify unnecessary synchronization and resource waits.
4. Check whether the scheduler efficiently selects runnable processes.

4. Optimization Techniques

1. Asynchronous I/O: Use asynchronous I/O and buffering where appropriate to reduce blocking time.

2. Improve scheduling: Choose a suitable scheduling algorithm to ensure ready processes receive CPU time efficiently.

3. Increase concurrency: Keep independent processes available for execution while other processes wait for I/O.

4. Reduce unnecessary blocking: Improve synchronization and resource management.

5. Expected Outcome

- Reduced CPU idle time.
- Improved process execution.
- Better CPU utilization.
- Increased overall system performance.

Conclusion

Reducing unnecessary blocking and scheduling runnable processes efficiently can improve CPU utilization. The actual cause should be confirmed from scheduler and I/O logs before selecting the optimization.

Q3. Memory Bottleneck Debugging

Question: A multithreaded program frequently encounters page faults. Identify the memory issue causing performance degradation and propose two optimizations.

1. Given Information

Memory component	Size
Code	5 MB
Heap	150 MB
Stack per thread	1 MB
Number of threads	25
Available physical RAM	2 GB

2. Memory Calculation

Total thread stack memory:

$$25 \times 1 = 25\text{ MB}$$

Total listed memory:

$$5 + 150 + 25 = 180\text{ MB}$$

The listed memory usage is approximately 180 MB, excluding other memory requirements and overheads.

3. Problem Identification

The program experiences frequent page faults, which can slow execution. Possible causes

include poor memory locality, memory pressure from other processes, excessive memory allocation, or an active working set that does not fit comfortably in available physical memory.

The given figures alone do not prove that the program exceeds physical RAM.

4. Root Cause Analysis

- Frequently accessed pages may not remain in physical memory.
- Repeated allocation and deallocation can increase memory-management overhead.
- Poor data locality can increase page faults.
- Other processes may compete for physical memory.
- Excessive thread creation may increase stack and scheduling overhead.

5. Two Optimization Techniques

Optimization 1: Improve memory management

- Release unused memory.
- Avoid unnecessary memory allocations.
- Reuse allocated objects when appropriate.
- Monitor heap usage and memory consumption.

Optimization 2: Improve memory locality

- Store related data close together.
- Access data in a sequential and predictable manner where possible.
- Reduce unnecessary memory accesses.
- Monitor page-fault frequency and working-set size.

6. Expected Outcome

- Reduced page faults.
- Lower memory-management overhead.
- Improved execution speed.
- Better overall system performance.

Conclusion

Memory-management improvements and better data locality can reduce page faults and improve performance. Memory and page-fault measurements should be examined to determine the exact bottleneck.

Q4. Starvation Debugging

Question: Process P5 never gets CPU time in a multilevel queue scheduler. Debug the root cause of starvation and suggest an optimization to ensure fairness.

1. Given Information

Queue	Priority	Scheduling method
Q1	Highest – Interactive	Round Robin, 5 ms quantum
Q2	System	Round Robin, 10 ms quantum
Q3	Lowest – Batch	FCFS

Process P5 belongs to Q3 but keeps getting postponed.

2. Problem Identification

Starvation occurs when a process waits indefinitely for CPU time because the scheduler repeatedly favors higher-priority processes.

3. Root Cause Analysis

- Q1 has the highest priority.
- Q2 has higher priority than Q3.
- New processes may continuously enter Q1 and Q2.
- The scheduler repeatedly serves higher-priority queues.
- P5 remains in Q3 without receiving sufficient CPU time.

4. Debugging Steps

1. Examine the scheduling logs for P5.
2. Measure how long P5 remains in the ready queue.
3. Check whether higher-priority queues continuously receive CPU time.
4. Verify whether the scheduler provides CPU time to lower-priority queues.

5. Optimization Techniques

Optimization 1: Aging

Gradually increase the priority of processes that wait for a long time. P5 can eventually receive a higher priority and CPU time.

Optimization 2: Priority Boosting

Periodically promote waiting processes from Q3 to a higher-priority queue.

Optimization 3: Fair CPU Allocation

Reserve a portion of CPU time for Q3 so that batch processes receive execution opportunities even when higher-priority queues are busy.

6. Expected Outcome

- Prevents indefinite waiting.
- Improves fairness among processes.
- Ensures P5 eventually receives CPU time.
- Improves overall scheduling efficiency.

Conclusion

Aging, priority boosting, and fair CPU allocation can prevent starvation. These methods ensure that lower-priority processes eventually receive CPU time without completely removing the priority structure.

Q5. Context-Switch Overhead Debugging

Question: A server performs 20,000 context switches per second, causing high overhead. Explain why context switching is high and propose two optimizations.

1. Given Information

Parameter	Value
Context switches	20,000 per second
Average burst time per thread	2 ms
I/O wait frequency	Very high
Scheduler	Preemptive
Time quantum	1 ms

2. Problem Identification

Context switching occurs when the CPU switches execution from one thread or process to another. A high context-switch rate can increase overhead and reduce the time available for useful work.

3. Root Cause Analysis

1. Short time quantum: The 1 ms quantum is shorter than the average 2 ms CPU burst. This can cause threads to be preempted before completing their bursts.

2. Frequent I/O waits: Threads frequently block while waiting for I/O, causing the scheduler to select other threads.

3. Scheduling overhead: Each context switch requires saving and restoring execution state.

4. Repeated thread scheduling: Frequent blocking and preemption increase scheduling activity.

4. Debugging Steps

1. Examine context-switch statistics.
2. Check how often threads are preempted after using the full quantum.
3. Measure I/O blocking frequency and duration.
4. Determine whether the workload needs a different scheduling policy.

5. Two Optimization Techniques

Optimization 1: Adjust the time quantum

Increase the time quantum to a suitable value based on the workload. This can reduce unnecessary preemption and allow CPU-bound threads to complete more work per scheduling interval.

However, an excessively large quantum may reduce responsiveness for interactive processes.

Optimization 2: Optimize I/O handling

- Use asynchronous I/O where appropriate.
- Apply buffering to reduce unnecessary waiting.
- Avoid unnecessary blocking operations.
- Improve I/O handling so threads can perform useful work while other operations are pending.

6. Expected Outcome

- Fewer unnecessary context switches.
- Reduced scheduling overhead.
- Better CPU efficiency.
- Improved server throughput and performance.

Conclusion

A suitable time quantum and efficient I/O handling can reduce context-switch overhead. The scheduler logs should be analyzed to confirm whether preemption, I/O blocking, or both are the main contributors.